FINAL ARCHITECTURE

Grass Content Factory

A content factory is a closed-loop system that turns ecosystem signals into ranked content opportunities, channel-native assets, measurable distribution, and feedback that improves the next cycle. This document is intended to make the framework, flow, and reasoning clear for an engineering team. It does not prescribe implementation details.

CORE THESIS

```
signals → opportunities → ranked packets → published assets → performance signals → strategy memory
```

Factory Loop

This is the high-level system. Each layer consumes one structured artifact and emits the next.

LAYER 0
Surface Registry
Decides what parts of the ecosystem matter right now.

→

LAYER 1
Scout
Pulls raw signals from official, community, demand, and market sources.

→

LAYER 2
Strategist
Turns signals into candidate content objects and ranks them.

→

LAYER 3
Creator
Builds the canonical asset and its native derivatives.

→

LAYER 4
Distributor
Publishes by channel grammar, voice type, and CTA style.

→

LAYER 5
Measurement
Measures performance, updates memory, and changes future priorities.

Design principle

The system should make better decisions over time. A content factory is not a publishing schedule. It is a learning loop.

Layer Contracts

The clearest way to explain the factory to engineers is to describe each layer by its input, responsibility, output, and decision.

LAYER	PRIMARY INPUT	RESPONSIBILITY	PRIMARY OUTPUT	DECISION
Surface Registry	Historical results, current signals, product strategy	Maintain active permanent and rotating surfaces	Active surface set	What ecosystem areas to watch
Scout	Active surfaces	Collect and cluster signals from multiple source classes	Signal clusters	What pain and momentum exist right now
Strategist	Signal clusters	Convert clusters into content objects and distribution packets	Ranked candidate packets	What to make next
Creator	Approved packet	Create canonical and derivative assets	Publish-ready assets	How to express the idea clearly
Distributor	Publish-ready assets	Publish per channel grammar and voice assignment	Live URLs, posts, and participation touchpoints	Where and how to distribute
Measurement	Live assets and channel performance	Track GEO, SEO, engagement, citations, and assist metrics	Performance scorecards	What worked and what failed
Strategy Memory	Performance scorecards	Write findings back into planning memory	Updated heuristics and registry state	What to reinforce, demote, or drop

Core Objects

The factory should operate on a small set of structured objects. This makes the reasoning explicit and keeps the system inspectable.

OBJECT 1
Surface
A tool, workflow, protocol, or infrastructure pattern worth watching.

OBJECT 2
Signal
A freshness, pain, demand, or market observation from scouting.

OBJECT 3
Content Object
A candidate topic expressed as a structured problem and outcome.

OBJECT 4
Distribution Packet
A bundle of channel-native assets built from one approved idea.

OBJECT 5
Scorecard
A performance record used to improve future scouting and strategy.

CONTENT OBJECT EQUATION

`surface → friction → outcome → format → Grass role`

```
Surface: Claude Code
Friction: session dies when terminal closes
Outcome: keep the agent running overnight
Format: troubleshooting
Grass role: light
```

```
Surface: remote agent runtime
Friction: local sessions are fragile
Outcome: choose the right persistence approach
Format: comparison
Grass role: evaluate
```

Surface Registry

Surfaces should be maintained as a live registry, not a fixed list. The registry keeps the factory focused on areas where Grass can build compounding authority.

Permanent Surfaces

Stable surfaces tied to durable developer jobs and repeated Grass value.

- session persistence
- remote agent runtime
- phone monitoring
- approval gates
- BYOK and key ownership
- multi-agent workflows

Rotating Surfaces

Current tools, launches, and ecosystem shifts reevaluated regularly.

- Claude Code
- Cursor cloud agents
- OpenAI Codex / Codex CLI
- Anthropic MCP
- Daytona
- new agent frameworks

Scoring Inputs

- product_fit
- pain_density
- demand
- proofability

Secondary Inputs

- distribution_fit
- freshness
- strategic_value

Momentum

Use recent signal volume versus baseline signal volume to detect what is rising or cooling.

Capacity

Keep the active set small so the factory can actually develop authority.

Portfolio rule

Use roughly 60% permanent surfaces and 40% rotating surfaces. The goal is to stay anchored to durable jobs while still exploiting ecosystem momentum.

Scout Layer

The scout does not write content. Its job is to turn messy ecosystem data into structured signal clusters and candidate opportunities.

BUCKET 1

Official Change Signals

- vendor docs and blogs
- product changelogs
- announcements and release notes

BUCKET 2

Community Pain Signals

- Reddit
- Hacker News
- GitHub issues and discussions
- community forums

BUCKET 3

Demand Signals

- Search Console
- Google Trends
- site search
- autosuggest and related queries

BUCKET 4

Market Framing

- competitor blogs
- launch posts
- comparison pages
- release roundups

Minimum Evidence Rule

- 1 pain signal
- 1 demand signal
- 1 freshness or proof signal

Scout Output

- surface
- friction
- outcome
- possible formats
- possible channels
- proof assets

Scope rule

Use parallel search by source class, not “search everywhere.” Too many sources create noise faster than insight.

Strategy Layer

The strategist turns signal clusters into candidate content packets, scores them, and chooses what to make.

M0_RESOLVE

Fix or understand a problem. Grass absent or incidental.

M1_EVALUATE

Compare solution paths. Grass present but not dominant.

M2_EXECUTE

Set up, operate, or optimize Grass directly.

Scoring dimensions

demand + proximity + proof + freshness + defensibility

Packet decision

Choose the highest-value topic and the best distribution packet underneath it.

```
if intent in {debug, understand}:
    mode = M0_RESOLVE

elif intent in {compare, choose}:
    mode = M1_EVALUATE

elif intent in {setup, operate,
optimize}:
    mode = M2_EXECUTE
```

Strategic unit

The strategist should rank packets, not just topics. One signal cluster can yield multiple formats and channels, but only some combinations are worth making.

Distribution Architecture

Distribution is not reposting. It is the design of native units across different channels, each serving a different job.

LAYER 1

Canonical Memory

Grass blog or docs as the updateable source of truth.

LAYER 2

Searchable Syndication

Medium, DEV, or Hashnode when canonical handling exists.

LAYER 3

Proof Layer

GitHub, gists, diagrams, screenshots, and terminal recordings.

LAYER 4

Conversation Layer

X, LinkedIn, Hacker News, and short narrative formats.

LAYER 5

Participation Layer

Reddit, issues, comments, replies, and live problem threads.

Distribution packet

1 canonical asset, 1 searchable syndication asset, 1 proof artifact, 2 native conversation assets, and 0–3 participation targets.

Voice assignment

brand_canonical, engineer_voice, founder_operator_voice, contributor_voice, or community_voice.

UGC-style rule

Use real first-person practitioner voice and transparent affiliation. Never fake neutrality.

```
{
  "canonical_asset": "Grass blog post",
  "syndication_target": ["DEV"],
  "proof_artifact": "GitHub gist + terminal screenshots",
  "native_units": ["X thread", "LinkedIn post"],
  "participation_targets": ["Reddit thread", "GitHub discussion"],
  "voice_type": "engineer_voice",
  "primary_goal": "citation"
}
```

Distribution rule

Do not distribute articles. Distribute native units that all point back to the same underlying insight and proof.

Measurement And Learning Layer

After publishing, the factory needs one place to evaluate whether the system is actually working and write those learnings back into planning.

Content-Level Metrics

- organic traffic and rankings
- engagement and retention
- referral clicks and assists
- direct conversion influence

GEO / Citation Metrics

- LLM citation checks
- AI search inclusion
- share of voice on target prompts
- extractability of canonical pages

Factory-Level Metrics

- surface hit rate
- packet efficiency by channel
- mode performance
- time-to-learning

Memory artifact

Store findings in a strategy-memory file or equivalent planning store.

Feedback action

Use findings to update surfaces, packet heuristics, proof preferences, and channel rules.

```
performance event
→ scorecard
→ strategy note
→ registry update
→ new scout focus
```

Measurement rule

The point of measurement is not reporting. It is decision improvement. The system should learn which surfaces, formats, proofs, voices, and channels compound best.

Operating Principles

These principles keep the factory useful, bounded, and strategically sound.

Start from the broken workflow, not the keyword
The best topics begin with real friction, not abstract category coverage.

Use multiple signal types before committing
No topic should be chosen from a single source alone.

Rank packets, not just articles
The system should choose the best expression of an idea across channels.

Keep one canonical memory layer
Grass should own the source-of-truth page even when discovery happens elsewhere.

Use native formats and real voices
Human trust comes from situated voice, proof, and channel fit, not from pretending to be neutral.

Close the loop
If learning does not update the registry, scouting, or packet rules, the system is incomplete.

Shortest version

Maintain a live surface registry, scout for multi-source signals, turn those signals into content objects, rank distribution packets, publish one canonical memory asset plus native derivatives, measure results, and write the findings back into strategy memory.